

Material de Apoio: Fundamentos de Numpy na Saúde

1. Introdução ao NumPy

O que é?

O NumPy é a principal biblioteca de computação numérica e matemática do Python.

Por que importar se já temos o Pandas?

Se o Pandas é o organizador de tabelas, o NumPy é o motor matemático dele. O Pandas foi construído em cima do NumPy.

Ele trabalha com **Arrays**, que são estruturas de alta performance (feitas na linguagem C, muito mais rápida que o Python). Por trás das cortinas, cada coluna do Pandas guarda seus números dentro de um Array do NumPy para conseguir processar os cálculos tão rápido.

Por convenção, importamos o NumPy com o apelido `np`:

```
In [ ]: import numpy as np
```

2. Arrays vs Listas

A **Lista do Python** é como uma maleta médica genérica: cabe gaze, seringa, esparadrapo (texto, número, booleanos). É flexível, mas muito lenta para a IA procurar e calcular coisas.

O **Array do NumPy** é como uma bandeja cirúrgica: só aceita um tipo de coisa (ex: apenas números). Essa rigidez permite que o computador processe milhares de operações de uma vez só (**Vetorização**).

```
In [ ]: lista = [1, 2, 3]
arr = np.array([1, 2, 3])

# Acessando e cortando (slicing)
arr[0]      # Retorna 1
arr[0:2]    # Retorna [1, 2]
```

```
Out[ ]: array([1, 2])
```

3. Arrays Especiais

Existem funções prontas do NumPy para criar arrays com valores pré-definidos, muito úteis para inicializar estruturas antes de preenchê-las com dados reais.

3.1 np.arange()

Gera uma sequência numérica espaçada, como uma régua.

```
In [ ]: np.arange(0, 10, 2)
# array([0, 2, 4, 6, 8])
```

```
Out[ ]: array([0, 2, 4, 6, 8])
```

3.2 np.zeros()

Cria matriz preenchida com zeros. Muito usado para inicializar pesos em IA.

```
In [ ]: np.zeros(3)
# array([0., 0., 0.])
```

```
Out[ ]: array([0., 0., 0.])
```

3.3 np.ones()

Semelhante ao zeros, mas preenche estruturas matemáticas com o número 1.

```
In [ ]: np.ones((2, 2))
# array([[1., 1.],
#        [1., 1.]])
```

```
Out[ ]: array([[1., 1.],
               [1., 1.]])
```

4. Matrizes multidimensionais e IA

Um array não precisa ser apenas uma linha de dados (1D). Ele pode formar tabelas e cubos.

```
In [ ]: m = np.array([[1, 2, 3],
                      [4, 5, 6]])

m.shape # Dimensões: (2, 3)
m.ndim  # Quantidade de dimensões: 2
```

```
Out[ ]: 2
```

4.1 A Ponte: De Pixels a Modelos

O computador não enxerga "ossos" ou "tecidos". Uma Radiografia Digital, Tomografia ou Mapa Raster é lida pela Inteligência Artificial apenas como uma gigantesca tabela de números (**Matriz NumPy**), onde cada número representa um tom de cinza de um pixel (0 a 255).

Observação para Gabriela: quando vc terminar você coloca esses comandos para converter para PDF (na dúvida, fala com Livia, ela quem me ensinou)

```
In [ ]: !apt-get update -qq
!apt-get install -y -qq texlive-xetex texlive-fonts-recommended texlive-p
```

```
In [ ]: !jupyter nbconvert --to html '/content/Pandas_para_saúde (1).ipynb'
```

5. Reshape

5.1 array.reshape()

- A função `.reshape()` permite remodelar as dimensões de uma matriz ou alterar o número de elementos em cada dimensão.
- É possível fazer o reshape de uma matriz desde que a matriz final seja uma matriz regular (todas as linhas precisam ter o mesmo número de elementos). Caso contrário, o código quebra.
- A quantidade de parâmetros passados na função define o número de dimensões da nova matriz.
- A ordem dos parâmetros vai da estrutura maior para o elemento individual: o primeiro número define a divisão mais externa, e o último define quantos elementos individuais existem em cada linha/coluna.
- Exemplo: em uma matriz 2D passamos (linhas, colunas). Se for 3D, vira (blocos, linhas, colunas), e assim por diante.

```
In [ ]: import numpy as np

array = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
print(array)
novoArray = array.reshape(4, 3)
print(novoArray)

# [ 1  2  3  4  5  6  7  8  9 10 11 12]
# [[ 1  2  3]
#   [ 4  5  6]
#   [ 7  8  9]
#   [10 11 12]]
```

5.2 Dimensão desconhecida (-1)

- Sempre calcular manualmente a dimensão apropriada ao usar a função `.reshape()` é bem trabalhoso. Por isso, podemos usar o que é chamado de "Dimensão desconhecida".
- Ao colocar `-1` como um dos argumentos, o próprio NumPy calcula automaticamente isso para você.

```
In [ ]: arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(3, -1)
print(newarr)

# [[ 1  2  3  4]
#   [ 5  6  7  8]
#   [ 9 10 11 12]]
```

```
In [ ]: arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(2, 3, -1)
print(newarr)

# [[[ 1  2]
#    [ 3  4]
#    [ 5  6]]
#
#   [[ 7  8]
#    [ 9 10]
#    [11 12]]]
```

5.3 -1 sozinho

Além disso, se o `-1` for colocado sozinho dentro da função, ele transforma a matriz em um array de apenas uma única dimensão.

```
In [ ]: array = np.array([[1,2,3],[4,5,6],[7,8,9]])
novoArray = array.reshape(-1)
print(array)
print(novoArray)

# [[1 2 3]
#   [4 5 6]
#   [7 8 9]]
# [1 2 3 4 5 6 7 8 9]
```

6. Funções Universais

As funções universais (ou *ufuncs*) são funções prontas oferecidas pelo NumPy para realizar operações vetorizadas nos arrays/matrizes.

Operações vetorizadas são uma forma de executar grandes cálculos matemáticos em arrays, matrizes e/ou blocos de dados de uma só vez sem precisar de laços de repetição.

As unfucs são utilizadas para fazer operações matemáticas e/ou lógicas de elemento a elemento.

As unfucs, ao serem chamadas, sempre criam um novo array como resultado da operação.

Para que a função seja realizada, os arrays e matrizes envolvidos precisam ter as mesmas dimensões/shapes.

6.1 Operações de Matemática Básica

Sendo X e Y arrays que possuem a mesma dimensão:

`np.add(x,y)` : Soma uma matriz a outra, elemento por elemento

`np.subtract(x,y)` : subtrai uma matriz da outra, elemento por elemento

`np.multiply(x,y)` : Multiplica os elementos, elemento por elemento

`np.divide(x,y)` : Divide os elementos, elemento por elemento

`np.power(x,y)`: Eleva um número a potência da outra

`np.mod(x,y)` : Retorna o resto da divisão, elemento por elemento

`np.sqrt(array)`: calcula raiz quadrada de cada elemento

```
In [ ]: a = np.array([4, 9, 16])
        b = np.array([2, 3, 4])

        print("Adição: ", np.add(a, b))
        print("Subtração: ", np.subtract(a, b))
        print("Multiplicação: ", np.multiply(a, b))
        print("Divisão: ", np.divide(a, b))
        print("Potência: ", np.power(a, b))
        print("Resto da divisão: ", np.mod(a, b))
        print("Raiz quadrada: ", np.sqrt(a))

        # Adição:  [ 6 12 20]
        # Subtração:  [ 2  6 12]
        # Multiplicação:  [ 8 27 64]
        # Divisão:  [2.  3.  4.]
        # Potência:  [  16  729 65536]
        # Resto da divisão:  [0 0 0]
        # Raiz quadrada:  [2.  3.  4.]
```

6.2 Arredondamentos e Valores Absolutos

`np.absolute(array)`: Transforma os números negativos em positivos (tira o sinal)

`np.floor(array)`: Arredonda para baixo (2.9 vira 2)

`np.ceil(array)` : Arredonda para cima (2.1 vira 3)

`np.round(array)` : Arredonda para o número mais próximo (2.1 vira 2 e 2.9 vira 3)

`np.trunc(array)`: Tira as casas decimais (2,43 vira 2)

`np rint(array)`: Arredonda pro número mais próximo, mas mantém como float (2.3 vira 2.0)

```
In [ ]: numeros = np.array([-2.7, 2.1, 3.8])

print("Valor absoluto:", np.absolute(numeros))
print("Arredonda para baixo (floor):", np.floor(numeros))
print("Arredonda para cima (ceil):", np.ceil(numeros))
print("Arredonda para o mais próximo (round):", np.round(numeros))
print("Trunca / remove decimais (trunc):", np.trunc(numeros))
print("Arredonda mantendo float (rint):", np.rint(numeros))

# Valor absoluto: [2.7 2.1 3.8]
# Arredonda para baixo (floor): [-3.  2.  3.]
# Arredonda para cima (ceil): [-2.  3.  4.]
# Arredonda para o mais próximo (round): [-3.  2.  4.]
# Trunca / remove decimais (trunc): [-2.  2.  3.]
# Arredonda mantendo float (rint): [-3.  2.  4.]
```

6.3 Comparações Lógicas entre Arrays

`np.greater(x, y)` : Verifica se o primeiro é maior que o segundo, elemento a elemento

`np.less(x, y)` : Verifica se o primeiro é menor que o segundo, elemento a elemento

`np.equal(x, y)`: Verifica se os dois são iguais, elemento a elemento

`np.not_equal(x, y)`: Verifica se os dois são diferentes, elemento a elemento (lógica invertida)

`np.logical_and(x, y)`: Retorna True apenas se ambos os elementos forem verdadeiros, elemento a elemento

`np.logical_or(x, y)`: Retorna True se pelo menos um dos elementos for verdadeiro, elemento a elemento

```
In [ ]: a = np.array([5, 10, 15])
b = np.array([3, 10, 20])

print("Maior que (greater):", np.greater(a, b))
print("Menor que (less):", np.less(a, b))
print("Igual a (equal):", np.equal(a, b))
print("Diferente de (not_equal):", np.not_equal(a, b))

print("-----")

v1 = np.array([True, True, False])
v2 = np.array([True, False, False])

print("E lógico (logical_and):", np.logical_and(v1, v2))
print("OU lógico (logical_or):", np.logical_or(v1, v2))
```

```
# Maior que (greater): [ True False False]
# Menor que (less): [False False  True]
# Igual a (equal): [False  True False]
# Diferente de (not_equal): [ True False  True]
# -----
# E lógico (logical_and): [ True False False]
# OU lógico (logical_or): [ True  True False]
```

7. Outras Funções Universais

7.1 Exponencial, Logaritmo e Trigonometria

`np.exp(a)` : calcula o exponencial (numero de euler elevado a elemento por elemento)

`np.log(a)` : calcula logaritmo natural (base e) de elemento a elemento

`np.log10(a)` : calcula logaritmo na base 10 de elemento a elemento

`np.log2(a)`: calcula logaritmo na base 2, elemento por elemento

`np.sin(a)` : calcula o seno do ângulo

`np.cos(a)` : calcula o cosseno do ângulo

`np.tan(a)`: calcula a tangente do ângulo

```
In [ ]: numeros = np.array([1.0, 10.0, 100.0])
        angulos = np.array([0, np.pi / 2, np.pi])

        print("Exponencial (e^x):", np.exp(numeros))
        print("Logaritmo natural (base e):", np.log(numeros))
        print("Logaritmo base 10:", np.log10(numeros))
        print("Logaritmo base 2:", np.log2(numeros))
        print("Seno:", np.sin(angulos))
        print("Cosseno:", np.cos(angulos))
        print("Tangente:", np.tan(angulos))
```

8. Concatenação

8.1 Concatenar Arrays

`np.concatenate(array)`: Junta dois ou mais arrays para formar um novo array.

Os 2 arrays precisam ter tamanhos compatíveis.

```
In [ ]: a = np.array([1, 2, 3])
        b = np.array([4, 5, 6])

        juncao = np.concatenate((a, b))
        print("Resultado:", juncao)

        # Resultado: [1 2 3 4 5 6]
```

9. Funções de Agregação

- Funções de agregação servem basicamente para pegar um conjunto com vários dados e resumi-los em um único valor.
- Normalmente, eles retornam apenas um valor unico, como um inteiro ou um float.
- No entanto, o parâmetro "axis" permite calcular por partes: axis=0 na vertical (por coluna) e axis=1 na horizontal (por linha), retornando um array com o resultado de cada uma em vez de apenas um único número.

9.1 np.sum, np.mean, np.min, np.max, np.argmin, np.argmax

np.sum(a) : soma todos os valores do array

np.mean(a) : Calcula a média aritmética do array

np.min(a): Retorna o valor minimo do array

np.max(a): Retorna o valor maximo do array

np.argmin(a): Diz o indice onde o valor minimo do array esta guardado

np.argmax(a): Diz o indice onde o valor maximo do array esta guardado

```
In [ ]: valores = np.array([10, 5, 40, 20, 15])

print("Soma:", np.sum(valores))
print("Média:", np.mean(valores))
print("Valor mínimo:", np.min(valores))
print("Valor máximo:", np.max(valores))
print("Índice do mínimo (argmin):", np.argmin(valores))
print("Índice do máximo (argmax):", np.argmax(valores))

# Soma: 90
# Média: 18.0
# Valor mínimo: 5
# Valor máximo: 40
# Índice do mínimo (argmin): 1
# Índice do máximo (argmax): 2
```

9.2 np.std e np.var

np.std(a): Calcula o desvio padrão, se o numero for grande, a maioria dos numeros não estão na media (o desvio padrão mede a regularidade das coisas e do quanto os valores variam, é tipo uma media do quão os numeros sai da media)

np.var(a): Variância (o desvio padrão ao quadrado): A diferença prática é que a variância dá um peso muito maior para valores que fogem muito do padrão (os extremos).

```
In [ ]: notas = np.array([6.0, 7.0, 6.5, 7.5, 7.0])

print("Desvio padrão (std):", np.std(notas))
print("Variância (var):", np.var(notas))

# Desvio padrão (std): 0.5099019513592784
# Variância (var): 0.25999999999999995
```

10. Valores Ausentes

- Em numpy existe um tipo de valor chamado "NaN" (Not a number, não é um número). Esse tipo de valor serve para representar espaços vazios.
- Ao usar funções de agregação com um array onde um NaN está presente, o valor final é alterado e terá o NaN como resultado na maioria das operações.
- Para resolver esse problema, ao utilizar uma função de agregação com a presença de um NaN, basta escrever um "nan" antes da função.

10.1 Funções nan*

`np.nansum(a)` : Calcula a soma dos elementos, ignorando os valores NaN

`np.nanmean(a)` : Calcula a média aritmética dos elementos, ignorando os valores NaN

`np.nanmedian(a)` : Calcula a mediana dos elementos, ignorando os valores NaN

`np.nanmax(a)` : Retorna o maior valor do array, ignorando os valores NaN

E assim por diante com as funções de agregação...

```
In [ ]: dados = np.array([10.0, np.nan, 20.0, 30.0, np.nan])

print("Soma sem NaN:", np.nansum(dados))
print("Média sem NaN:", np.nanmean(dados))
print("Mediana sem NaN:", np.nanmedian(dados))
print("Maior valor sem NaN:", np.nanmax(dados))

# Soma sem NaN: 60.0
# Média sem NaN: 20.0
# Mediana sem NaN: 20.0
# Maior valor sem NaN: 30.0
```

10.2 np.isnan()

Também existe uma função que analisa elemento por elemento e retorna um array, onde coloca True nas posições em que tem um NaN presente. O `.isnan(array)`

```
In [ ]: notas = np.array([7.5, np.nan, 9.0, np.nan, 4.0])
posicoes_vazias = np.isnan(notas)
print(posicoes_vazias)
```

```
# [False True False True False]
```

11. Indexação/Comparações Booleanas

É possível comparar e filtrar arrays usando operações booleanas. A comparação gera um array de True e False e a Indexação utiliza esse resultado como filtro para criar um novo array contendo apenas os valores desejados.

11.1 Comparação booleana

Para comparar, faça uma comparação normal e será gerado um array com True ou False dependendo da condição.

```
In [ ]: idades = np.array([15, 20, 12, 22])
        mascara = idades >= 18

        print(idades)
        print(mascara)

        # [15 20 12 22]
        # [False True False True]
```

11.2 Indexação booleana

Para indexar, coloque a condição entre colchetes depois do nome do array.

```
In [ ]: idades = np.array([15, 20, 12, 22])
        filtrado = idades[idades >= 18]

        print(idades)
        print(filtrado)

        # [15 20 12 22]
        # [20 22]
```

12. Pandas e NumPy

O Pandas foi criado usando o NumPy como base, ou seja, ao utilizar o Pandas, na verdade estamos usando o NumPy com uma roupagem diferente.

Enquanto Numpy é pensando mais para dados numericos brutos e só aceita um tipo de valor no array, o Pandas aceita multiplos tipos de valores, é focado em tabelas e em leitura de arquivos.

Na realidade, muitas bibiliotecas do python usa o NumPy como base.

```
In [ ]: import pandas as pd

        dados = [10, 20, np.nan, 40]
        coluna_pandas = pd.Series(dados)
```

```
array_escondido = coluna_pandas.to_numpy()

print(coluna_pandas)
print("-----")
print(array_escondido)

# 0    10.0
# 1    20.0
# 2     NaN
# 3    40.0
# dtype: float64
# -----
# [10. 20. nan 40.]
```